

MODULE 16 · DOMAIN 3 - DATA OPERATIONS AND SUPPORT

Cost and performance

The qualifier that decides one question in three, consolidated across every service, plus the cost tooling itself.

DOMAIN 3 of 4 EXAM WEIGHT 22% MODULE 16 of 27

In the bank analysed for this course,
roughly **one question in three** was decided
by the qualifier, not the service.

What each qualifier actually asks

PHRASE	WHAT IT ELIMINATES
LEAST operational overhead	Anything you would run, patch, size or babysit
LEAST development effort	Anything requiring custom code written once
MOST cost-effective	Anything cheaper that misses another stated requirement
LEAST latency	Anything that batches, buffers or schedules
Minimal changes to existing	Anything requiring a rewrite or a re-platform
Without disrupting operations	Anything that loads a production system

ON THE EXAM

Most cost-effective never means cheapest. It means cheapest among the options that satisfy everything else.

Cost follows the **billing unit**. Learn what a service charges for and the arithmetic follows.

What each service charges for, part one

SERVICE	BILLING UNIT
Amazon S3	Storage per GB, requests, retrieval from cold classes
AWS Glue ETL	DPU-hours while running, per second, one minute minimum
Amazon EMR on EC2	Instance hours for the whole cluster, running or idle
EMR Serverless	Per second per worker while a job runs
AWS Lambda	Requests, plus duration times configured memory

ON THE EXAM

Sort each row into two piles as you read: billed while idle, and billed only when working.

What each service charges for, part two

SERVICE	BILLING UNIT
Amazon Athena	Bytes scanned, or DPU-hours on a capacity reservation
Redshift provisioned	Node hours, plus managed storage separately
Redshift Serverless	RPU-seconds while active, plus storage
Kinesis Data Streams	Shard hours, or per GB on demand
Amazon Data Firehose	Per GB ingested
Amazon DynamoDB	Per request, or provisioned capacity units per hour

ON THE EXAM

Redshift bills compute and storage independently. Shrinking a cluster does nothing to a storage bill.

Real optimisation makes the system **do less work**. It does not buy the same work cheaper.

The levers, by where they act

LEVER	WHERE IT APPLIES
Read less data	Partitioning, columnar formats, pushdown predicates, SELECT column lists
Process less data	Bookmarks, incremental loads, filtering early, CDC instead of full loads
Repeat less work	Materialized views, result reuse, SPICE, caching
Hold less capacity	Transient clusters, pause and resume, serverless, auto scaling
Store more cheaply	Lifecycle rules, Intelligent-Tiering, archive classes, log retention
Commit to usage	Reserved Instances, Savings Plans, Spot for interruptible work

ON THE EXAM

The first five reduce work. Only the last one buys the same work cheaper, and it is the smallest lever of the six.

Cheaper hardware for the same work

- ◆ **Graviton processors** deliver better price performance on the same workload.
- ◆ **Redshift RG instances** are Graviton based, and thirty percent cheaper per vCPU than RA3.
- ◆ **Lambda on arm64** costs less per gigabyte-second than x86.
- ◆ **EMR, OpenSearch and RDS** all offer Graviton instance types.
- ◆ **Check compatibility** for anything with native dependencies compiled for x86.

ON THE EXAM

This is the rare lever that reduces cost without reducing work or committing to a term.

Data transfer, the cost nobody plans for

MOVEMENT	CHARGED?
Into AWS from the internet	Free
Within one Availability Zone	Free
Between Availability Zones in a Region	Charged in both directions
Between Regions	Charged, and more than cross-AZ
Out to the internet	Charged, beyond a monthly free allowance
To S3 or DynamoDB through a gateway endpoint	Free, and it avoids NAT gateway charges

ON THE EXAM

The last row is the practical one. Routing S3 traffic through a NAT gateway instead of a gateway endpoint is a pure waste.

A company's analytics spend is dominated by three things. A nightly Glue job that reprocesses all historical data because bookmarks were never enabled. Athena queries over uncompressed CSV in a single unpartitioned prefix. And an EMR cluster running continuously for a job that takes ninety minutes each night.

Finance has asked for a substantial reduction. The team has been offered a Reserved Instance discount on the EMR cluster and is considering taking it.

What should the data engineer prioritise?

- ☐ A Purchase the Reserved Instances, which discount the largest single line item
- ☐ B Enable bookmarks, convert to partitioned Parquet, and switch EMR to a transient cluster
- ☐ C Move the S3 data to Standard-Infrequent Access to reduce storage cost
- ☐ D Set an AWS Budget so the team is alerted when spend exceeds the target

◆ PAUSE AND DECIDE

What should the data engineer prioritise?

A Buy the Reserved Instances
Commits for one or three years to a cluster that should be running ninety minutes a night. It discounts waste.

C Move to Standard-IA
Reduces storage cost and adds retrieval charges to actively queried data. Athena bills bytes scanned regardless.

B Bookmarks, Parquet and partitioning, transient cluster
Each removes work rather than discounting it. Reprocessing stops, scanning collapses, and idle hours disappear.

D Set a budget
Tells you the bill is high. It is a visibility control, not a reduction, and they already know.

THE SIGNAL

Buying a commitment for a badly designed workload locks in the waste for one to three years.

Provisioned or serverless is the one decision the exam guide names **explicitly**.

Both forms, service by service

SERVICE	PROVISIONED FORM AND SERVERLESS FORM
Amazon EMR	EMR on EC2, or EMR Serverless
Amazon Redshift	Provisioned cluster with RG or RA3 nodes, or Redshift Serverless
Amazon MSK	MSK Provisioned, or MSK Serverless
Kinesis Data Streams	Provisioned shards, or on-demand
Amazon Aurora	Provisioned instances, or Aurora serverless
Amazon DynamoDB	Provisioned capacity, or on-demand
Amazon MWAA	Provisioned environment, or MWAA Serverless
Amazon OpenSearch	Managed domain, or OpenSearch Serverless
Amazon Athena	On-demand per query, or capacity reservations

ON THE EXAM

Athena is the interesting inversion. Its default is pay per use, and the reservation is the provisioned option.

What actually decides it

PROVISIONED WINS

Steady and predictable

- High, constant utilisation around the clock
- A measured throughput figure in the question
- Commitment discounts available on top
- Fine-grained tuning genuinely needed

SERVERLESS WINS

Variable or occasional

- Spiky, seasonal or unknown demand
- Long idle periods between bursts
- Nobody has measured it yet
- Operational overhead is the stated qualifier

THE TIEBREAKER

A stated throughput number usually means provisioned. The words unpredictable, variable or unknown mean serverless.

Where the intuition fails

- ◆ **Serverless is not always cheaper.** At sustained high utilisation it is usually dearer.
- ◆ **Provisioned is not always cheaper either.** Idle capacity is pure waste.
- ◆ **Pausing changes the arithmetic.** A paused Redshift cluster costs no compute.
- ◆ **Auto scaling narrows the gap** by making provisioned capacity less fixed.
- ◆ **Minimums matter.** OpenSearch Serverless and MSK Provisioned both have floors.

ON THE EXAM

The crossover point is real. Below some utilisation serverless is cheaper; above it, provisioned is.

A company runs two Amazon Redshift environments. Production serves around three hundred analysts continuously through the working day, with a measured and stable workload. Development is used by four engineers for a few hours, two or three days a week, and its usage pattern is irregular.

Both currently run identically sized provisioned clusters, on demand, twenty four hours a day. The company wants to reduce cost without affecting production performance.

What should the data engineer recommend?

- ☐ A Move both environments to Redshift Serverless
- ☐ B Keep production provisioned with Reserved Instances, and move development to Redshift Serverless
- ☐ C Keep both provisioned and purchase Reserved Instances for both
- ☐ D Move production to Redshift Serverless and keep development provisioned

◆ PAUSE AND DECIDE

What should the data engineer recommend?

- A

 Serverless for both
Right for development, and production is steady and measured, which is exactly where reserved provisioned capacity is cheaper.
- B

 Reserved provisioned production, serverless development
Each environment goes to the side of the crossover it actually sits on. Steady and measured against irregular and idle.
- C

 Reserved for both
Commits for years to a development cluster used six hours a week. It discounts idle time.
- D

 Serverless production, provisioned development
Exactly backwards. It puts each workload on the wrong side of the trade.

THE SIGNAL

Two workloads, two answers. The instinct to standardise on one model is the wrong answer for the third time in this course.

Committing to usage

- ◆ **Reserved Instances** for Redshift, RDS, OpenSearch and EC2. One or three years.
- ◆ **Savings Plans** for compute, including Lambda and Fargate, with more flexibility.
- ◆ **Redshift Serverless reservations** exist too, so serverless is not always on demand.
- ◆ **Spot Instances** for interruptible work. EMR task nodes, as module five covered.
- ◆ **Commit only to a baseline** you are confident about, not to a peak.

ON THE EXAM

Commit to the floor of your usage, not the ceiling. Uncommitted burst can run on demand.

You cannot optimise what you cannot attribute. **Tagging comes before tuning.**

Cost allocation tags

- ◆ **A tag becomes a cost dimension** once activated in the billing console.
- ◆ **Then filter and group by it** in Cost Explorer, Budgets and the usage reports.
- ◆ **Activation is not retroactive.** Tags apply from the point you activate them.
- ◆ **AWS-generated and user-defined** tags are both available.
- ◆ **An untagged resource** is unattributable, and untagged spend accumulates quietly.

ON THE EXAM

Tagging tells you where the money goes. It does not reduce anything. That distinction appears in questions.

AWS Cost Explorer

- ◆ **Visualise spend** by service, account, Region, usage type or tag.
- ◆ **Around a year of history**, with forecasting ahead.
- ◆ **Usage type is the granular dimension**, distinguishing storage from requests from transfer.
- ◆ **Rightsizing recommendations** based on observed utilisation.
- ◆ **An API**, so cost data can feed a dashboard or a report.

ON THE EXAM

Grouping by usage type is what turns "S3 costs a lot" into "requests cost a lot, and that means small files".

AWS Budgets and anomaly detection

- ◆ A **budget** sets a threshold on cost or usage, with alerts at chosen percentages.
- ◆ **Filter a budget by tag** so each team owns its own threshold.
- ◆ **Budget actions** can apply a policy or stop resources when a threshold is crossed.
- ◆ **Cost Anomaly Detection** learns spend patterns and alerts on deviation.
- ◆ A **budget catches expected growth exceeding a target**. Anomaly detection catches the unexpected.

ON THE EXAM

The same distinction as module fourteen. A threshold catches what you predicted; anomaly detection catches what you did not.

Where the rest of the cost data lives

- ◆ **Cost and Usage Reports** deliver line-item detail to Amazon S3.
- ◆ Query them with **Athena**, which is a data engineering job like any other.
- ◆ Far more granular than **Cost Explorer**, down to individual resource and hour.
- ◆ **Cost Optimization Hub** consolidates recommendations across accounts, at no charge.
- ◆ **Cost Categories** group spend by business rules rather than by AWS structure.

ON THE EXAM

The usage report is a dataset in S3. Analysing your own bill is a pipeline you already know how to build.

A platform team supports twelve product teams sharing one AWS account for analytics. The monthly bill has grown forty percent in six months. Leadership wants each team held accountable for its own spend, and wants to know within a day if any team's costs jump unexpectedly.

Resources are currently untagged. The team has proposed setting a single account-wide budget alert at the current spend level.

What should the data engineer implement?

- ☐ A The account-wide budget alert as proposed
- ☐ B Apply and activate cost allocation tags per team, create per-tag budgets, and enable Cost Anomaly Detection monitors scoped to those tags
- ☐ C Move each team into its own AWS account
- ☐ D Use Cost Explorer weekly to review spend by service

◆ PAUSE AND DECIDE

What should the data engineer implement?

- A

 One account-wide budget
Tells you the account is over budget and nothing about which of twelve teams caused it.
- B

 Tags, per-tag budgets, anomaly monitors per tag
Attribution first, then accountability per team, then detection of the unexpected within a day.
- C

 An account per team
A real approach to attribution, and a large re-platforming project for a reporting requirement.
- D

 Weekly Cost Explorer review
Groups by service, not by team, on untagged resources. And weekly does not meet within a day.

THE SIGNAL

Three requirements: per-team attribution, accountability, and detection within a day.
Only one option covers all three.

Where the money usually is

- ◆ **Idle provisioned capacity.** Clusters running overnight for nobody.
- ◆ **Reprocessing.** Jobs re-reading all history because bookmarks were never enabled.
- ◆ **Scanning too much.** Unpartitioned or row-format data, queried repeatedly.
- ◆ **Storage nobody expires.** Noncurrent versions, logs, incomplete multipart uploads.
- ◆ **Defaults nobody changed.** Ten DPUs, never-expire retention, oversized nodes.

ON THE EXAM

The last row is the most common and the cheapest to fix. Somebody simply has to look.

Optimisations that make things worse

THE CHANGE	WHY IT BACKFIRES
Cheaper storage class for hot data	Retrieval charges on frequently read data exceed the storage saving
Archive tier for small objects	128 KB minimum billing multiplies the effective volume
Lifecycle transition before the minimum	Early deletion charges bill the remainder of the minimum anyway
Halving Lambda memory on CPU-bound work	Halves the CPU too. Duration doubles and cost barely moves
Reserved capacity for a badly designed workload	Locks in the waste for one to three years
Smaller Redshift cluster for a storage bill	Compute and managed storage are billed independently

ON THE EXAM

Every row here is an optimisation somebody made in good faith. Five of the six appeared as distractors earlier in this course.

Cost across account boundaries

- ◆ **Consolidated billing** aggregates accounts under AWS Organizations.
- ◆ **Volume discounts and commitments** apply across the whole organisation.
- ◆ **Cost Explorer filters by linked account**, which gives attribution without tags.
- ◆ **Cross-Region data transfer** is charged and easy to incur accidentally.
- ◆ **Cross-account data sharing** can shift cost to the consumer, or not, depending on design.

ON THE EXAM

An account boundary is the cleanest cost attribution there is. Tags are what you use when you cannot have one.

Cost and performance usually move together. When they do not, the question says which one matters.

Performance levers, consolidated

SYMPTOM	THE LEVER
Reading far more than needed	Partition, project columns, push predicates down
One worker busy, others idle	Fix the key. Skew is never a capacity problem
Slow before any data is read	Too many partitions, or millions of small files
Queries queue rather than run slowly	Concurrency scaling, WLM priority, capacity reservations
Repeated identical work	Materialized views, result reuse, SPICE
Cannot parallelise at all	Non-splittable compression, or a single input file

ON THE EXAM

Only one of these six rows is solved by adding capacity, and it is not the one people reach for.

An Athena workload costs around forty thousand dollars a month. Investigation shows the data is already Parquet and partitioned by date, queries filter on date, and most queries name specific columns. Roughly two hundred and forty thousand queries run per month, largely from scheduled dashboards refreshing every five minutes against data that updates hourly.

The team has already applied the standard optimisations and wants a further reduction.

What should the data engineer do next?

- ☐ A Convert the data to ORC instead of Parquet
- ☐ B Enable query result reuse, and align dashboard refresh to the hourly data update
- ☐ C Repartition the data by hour to improve pruning further
- ☐ D Move the workload to an Amazon Redshift provisioned cluster

◆ PAUSE AND DECIDE

What should the data engineer do next?

A ORC instead of Parquet

Both are columnar with similar characteristics. This is a marginal change dressed up as an optimisation.

C Repartition by hour

Queries filter by date, which daily partitions already serve. This adds partitions nobody filters on.

B Result reuse, and refresh at the data's actual cadence

Eleven of every twelve refreshes query unchanged data. Removing redundant work is the remaining lever.

D Move to Redshift

Defensible at this query volume, and it replaces the platform rather than fixing the waste inside it.

THE SIGNAL

Five minute refreshes against hourly data means eleven of every twelve queries recompute an unchanged answer.

Right-sizing, as a practice

- ◆ **Measure before changing.** Cost Explorer and CloudWatch show actual utilisation.
- ◆ **Change one thing at a time,** so you can attribute the effect.
- ◆ **Defaults are rarely right.** Ten DPUs on Glue, never-expire on log groups.
- ◆ **Review after growth,** because a size chosen last year fits last year's volume.
- ◆ **Rightsizing recommendations** in Cost Explorer are a starting point, not an instruction.

ON THE EXAM

Module fourteen's observation applies to cost as well. A surprising share of waste is a default nobody changed.

Reviewing an architecture deliberately

- ◆ **The AWS Well-Architected Tool** reviews a workload against defined pillars.
- ◆ **Cost optimisation is one pillar**, alongside performance efficiency and five others.
- ◆ **It produces a list of findings**, which is a plan rather than an opinion.
- ◆ **The Data Analytics Lens** applies the framework to analytics workloads specifically.
- ◆ On the in-scope service list, and rarely studied.

ON THE EXAM

A structured review catches the things nobody thought to look for. That is its whole value over an ad hoc audit.

Comparing two options under time pressure

- ◆ **Compare the shape, not the arithmetic.** The exam rarely needs a calculation.
- ◆ **Idle time is the fastest signal.** Long idle favours per-use billing decisively.
- ◆ **Volume of work is the other.** Sustained heavy load favours reserved capacity.
- ◆ **If two options differ by a few percent,** the question is testing something else.
- ◆ **Look for the option that removes a component,** which usually wins on both axes.

ON THE EXAM

If you find yourself doing serious arithmetic, re-read the question. The intended distinction is usually structural.

The order you read a question in

- ◆ Find the **qualifier first**, before reading a single option.
- ◆ Find the **stated refusals**. No new code, no source changes, no capacity to operate.
- ◆ Eliminate on those, which usually removes two options in seconds.
- ◆ Reading the **options first** forms an opinion you then look for reasons to justify.
- ◆ Then compare the **survivors** on their technical merits.

ON THE EXAM

Every worked scenario in this course was built so this order works, because that is how the exam builds them.

Reading a cost or performance question quickly

- ◆ **Find every constraint first.** Cost is compared only among options that satisfy them.
- ◆ **Which billing unit?** Idle-billed and work-billed have opposite answers.
- ◆ **Is work being repeated or wasted?** Removing it beats discounting it.
- ◆ **Measured or unpredictable?** That decides provisioned against serverless.
- ◆ **Is this attribution or reduction?** Tags and budgets tell you; they do not save.

ON THE EXAM

Domain 3 complete. Next: Domain 4 begins with authentication and network access.